FOORAN

Potentiometer-Based PWM Speed and LED Brightness Control Using STM32F103RB

STM32F103RB Microcontroller Hardware Abstraction Layer (HAL)

Prepared By: Ahsan Basharat | Status: Intern | Submitted To: Sir Yasir

Objectives

- To understand the basic concept and working principle of PWM (Pulse Width Modulation) for controlling output power.
- To configure the TIM3 peripheral of the STM32F103RB for PWM generation using the STM32 HAL Library.
- To interface a potentiometer with the ADC and convert its analog value into a PWM duty cycle.
- To control LED brightness and DC motor speed by varying the PWM duty cycle.
- To gain hands-on experience in developing, programming, debugging, and verifying ADC and PWM functionality using STM32CubeIDE and STM32F103RB.

Introduction

PWM (Pulse Width Modulation) is a technique used to control the average power supplied to electrical devices by varying the duty cycle of a digital signal. In this project, the STM32F103RB generates PWM signals using its timer peripheral to control LED brightness and DC motor speed. A potentiometer is used with the ADC to vary the PWM duty cycle.

PWM

In this experiment, PWM (Pulse Width Modulation) is generated using the TIM3 timer of the STM32F103RB. A potentiometer provides an analog input to the ADC through PA0. The ADC value is converted into a PWM duty cycle from 0% to 100%. The PWM signal is then used to control the brightness of three LEDs connected to PA6, PA7, and PB0, and can also be used to control the speed of a DC motor.

Required Components

#	Component / Tool	Quantity	Description / Specifications
1	STM32F103RB	1	ARM Cortex-M3 (32-bit)
2	USB Cable / ST-Link V2	1	Hardware programmer and debugger
3	Breadboard	1	Standard solderless prototyping breadboard
4	Light Emitting Diode (LEDS)	3	5mm Red/Green LED indicator
5	Jumper Wires	Set	Male-to-Male and Male-to-Female wires
6	STM32CubeIDE/MX	Software	Integrated Development Environment for C/C++

Time Clock

The timer clock is the clock signal that drives the STM32 timer counter. In this experiment, TIM3 uses the timer clock to count continuously and generate the required PWM signal. The timer clock frequency affects the resolution and frequency of the PWM signal.

$$\text{TIM CLOCK} = \frac{\text{APB TIM CLOCK}}{\text{PRESCALAR}}$$

APB (Advanced Peripheral Bus) is a bus inside the STM32 that provides the clock to peripherals such as TIM2, TIM3, USART, ADC, etc.

Frequency

The frequency is the number of complete PWM cycles generated per second and is measured in Hertz (Hz). In this experiment, TIM3 generates a PWM frequency of approximately 1 kHz, meaning 1000 PWM cycles are generated every second.

$$\text{FREQUENCY} = \frac{\text{TIM CLOCK}}{\text{ARR}}$$

The Prescaler divides the timer clock before it reaches the timer counter.

Formula:

$$Timer\ Clock = \frac{APB\ Timer\ Clock}{PSC + 1}$$

Duty Cycle

The duty cycle represents the percentage of one PWM period for which the signal remains ON. It is calculated using:

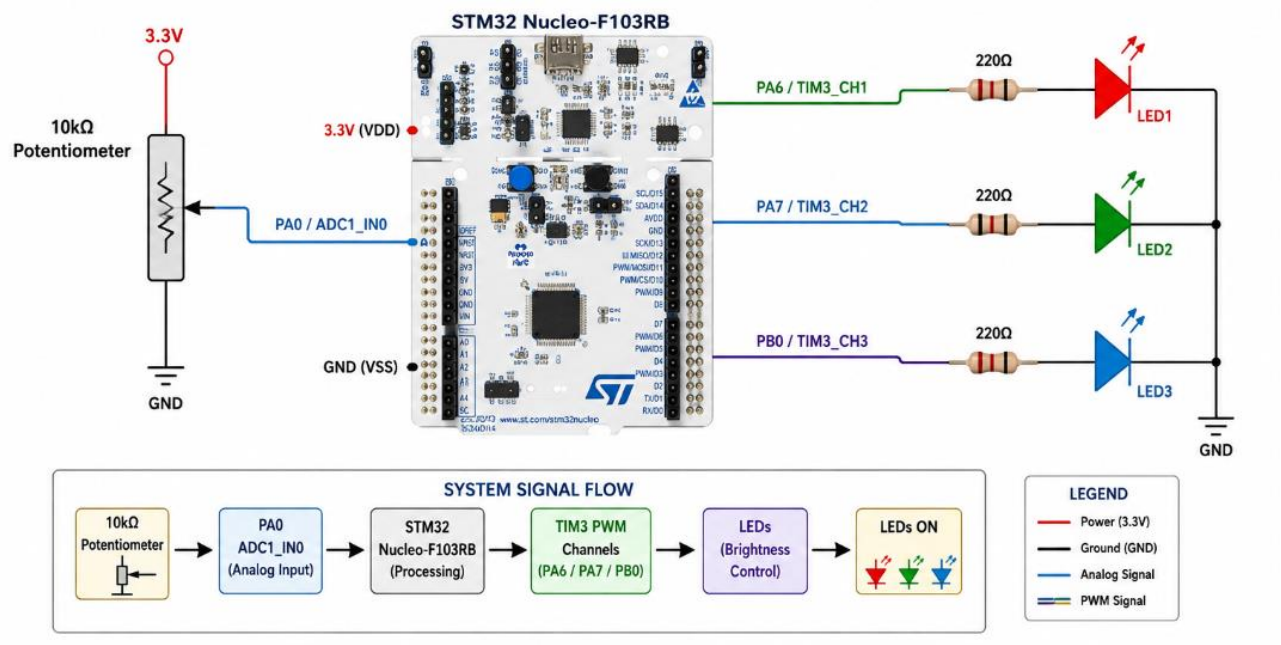
$$\text{DUTY \%} = \frac{\text{CCRx}}{\text{ARR}} \text{ X 100}$$

In this experiment, the duty cycle can be varied from 0% to 100% to control the brightness of the LEDs and the speed of the DC motor.

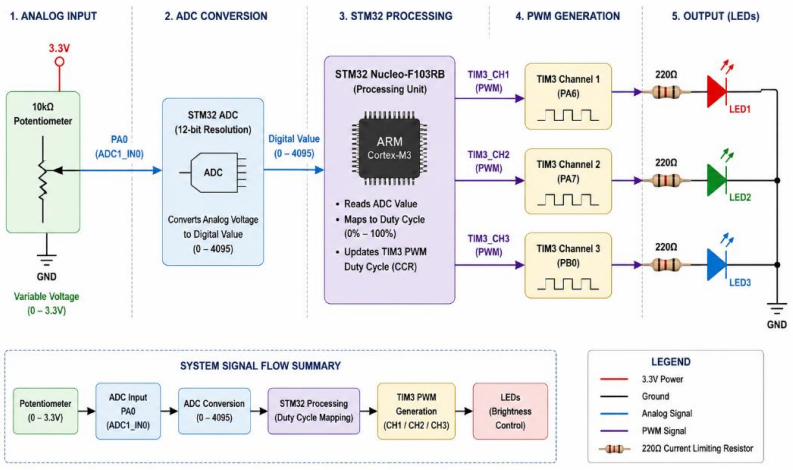
CCR (Capture/Compare Register) determines how long the PWM output stays HIGH/ON during each period.

ADC-Based PWM LED Control Circuit

This circuit demonstrates ADC-based PWM control using the STM32 Nucleo-F103RB. A 10kΩ potentiometer connected to PA0 (ADC1_IN0) provides a variable 0–3.3 V input. The 12-bit ADC converts it into a value from 0 to 4095, which is mapped to a 0–100% PWM duty cycle. The PWM signals are generated by TIM3 through PA6, PA7, and PB0 to control three LEDs. As the potentiometer is rotated, the duty cycle changes, adjusting the LED brightness from OFF at 0% to maximum brightness at 100%.



ADC-BASED PWM LED CONTROL FLOW GRAPH



The flow graph shows the complete working process of the ADC-based PWM LED control system. The 10kΩ potentiometer provides a variable voltage from 0 to 3.3 V to PA0 (ADC1_IN0). The STM32 ADC converts this analog voltage into a 12-bit digital value from 0 to 4095. The microcontroller then maps this value to a PWM duty cycle from 0% to 100%. Finally, TIM3 PWM channels generate signals through PA6, PA7, and PB0 to control the brightness of the three LEDs. As the potentiometer is rotated, the ADC value and PWM duty cycle change, resulting in a corresponding change in LED brightness.

Procedure: PWM Generation on STM32 Nucleo-F103RB (LED Brightness Control)

1. Launch **STM32CubeIDE** and create a new project by selecting the **STM32F103RB** microcontroller.
2. Open the **.ioc** configuration file and enable the required peripherals:
   - **ADC1** → In Single Conversion mode
   - **TIM3** → PWM Generation CH1, CH2 and CH3
3. Configure the **ADC1** with the following settings:
   - **Resolution**: 12 Bits
   - **Data Alignment**: Right
   - **Scan Conversion Mode**: Disable
   - **Continuous Conversion Mode**: Enable
   - **External Trigger**: Software Start
   - **Channel**: ADC1_IN0 (PA0)
4. Configure **TIM3** for **PWM Generation** with the following settings:
   - **Prescaler (PSC):** e.g., 71 (for 1 MHz timer clock if APB1 = 72 MHz)
   - **Counter Mode:** Up
   - **Period (ARR):** e.g., 999 (for 1 kHz PWM frequency)
   - **Clock Division:** DIV1
   - **PWM Mode:** PWM mode 1
   - **Enable Channels:** CH1 (PA6), CH2 (PA7), CH3 (PB0)
5. Configure GPIO pins for PWM output:
   - **PA6** → TIM3_CH1 (Alternate Function Push-Pull)
   - **PA7** → TIM3_CH2 (Alternate Function Push-Pull)
   - **PB0** → TIM3_CH3 (Alternate Function Push-Pull)
6. Save the IOC file and allow STM32CubeIDE to generate the initialization code.
7. In the user code, start the peripherals:
   - Start ADC1
   - Start TIM3 in PWM mode
8. Write the program to:
   - Read the ADC value from PA0.
   - Map the ADC value (0–4095) to **PWM** duty cycle (0–100%).
   - Update TIM3 CCR values for CH1, CH2 and CH3 to control LED brightness.
9. Build the project and program the STM32F103RB using the **ST-Link V2** debugger.
10. Adjust the potentiometer connected to PA0 and observe the change in brightness of the three LEDs connected to **PA6, PA7** and **PB0**.

Note: Ensure common GND between STM32 and all external components. Use 220Ω current limiting resistors with each LED.

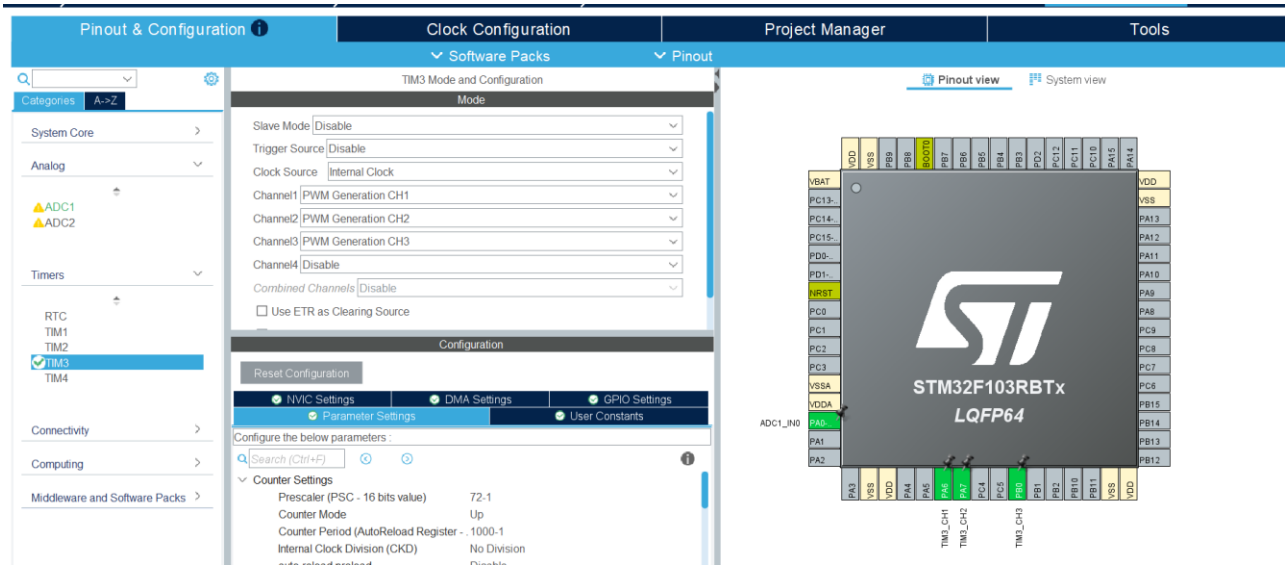
HAL Functions Used in PWM Generation (ADC-based LED Brightness Control)

Peripheral	HAL Function	Purpose / Description	When Used
ADC	HAL_ADC_Init(&hadc1)	Initializes ADC1 peripheral with configured parameters.	During initialization
	HAL_ADC_Start(&hadc1)	Starts the ADC conversion.	Before reading ADC value
	HAL_ADC_PollForConversion(&hadc1, HAL_MAX_DELAY)	Waits until ADC conversion is complete (polling).	After starting ADC
	HAL_ADC_GetValue(&hadc1)	Returns the converted digital value (0–4095).	After conversion complete
	HAL_ADC_Stop(&hadc1)	Stops the ongoing ADC conversion.	(Optional) When stopping ADC
TIMER (TIM3)	HAL_TIM_PWM_Init(&htim3)	Initializes TIM3 for PWM generation.	During initialization
	HAL_TIM_PWM_ConfigChannel(&htim3, &sConfig, TIM_CHANNEL_X)	Configures PWM channel (CH1/CH2/CH3) with mode, pulse, polarity.	During initialization (for each channel)
	HAL_TIM_PWM_Start(&htim3, TIM_CHANNEL_X)	Starts PWM signal on specified channel.	After TIM3 initialization
	__HAL_TIM_SET_COMPARE(&htim3, TIM_CHANNEL_X, value)	Updates PWM duty cycle by setting CCR value (0 to ARR).	In main loop (to change brightness)
	HAL_TIM_Base_Start(&htim3)	Starts TIM3 base counter.	(Optional) If not started automatically
GPIO	HAL_GPIO_Init(GPIOx, &GPIO_InitStruct)	Initializes GPIO pins (PA6, PA7, PB0) in Alternate Function Push-Pull mode for PWM.	During initialization (Generated by CubeMX)
SYSTEM	HAL_Init()	Initializes the HAL Library.	At the beginning of main()
	SystemClock_Config()	Configures the system clock.	At the beginning of main()
	HAL_Delay(ms)	Provides a delay in milliseconds (if used).	(Optional) For timing/debug

Note:

- TIM_CHANNEL_X → Use TIM_CHANNEL_1 for PA6 (CH1), TIM_CHANNEL_2 for PA7 (CH2), TIM_CHANNEL_3 for PB0 (CH3).
- ADC value (0–4095) is mapped to PWM duty cycle (0–ARR) using: CCR = (ADC_Value * (ARR + 1)) / 4095
- Prescaler (PSC) and ARR are set in CubeMX (e.g., PSC = 71, ARR = 999 for 1 kHz PWM with 72 MHz timer clock on APB1).

Code:




```
typedef struct
{
    TIM_HandleTypeDef *TimerHandle;
    uint32_t Channel;
    uint32_t Period;

} PWM_HAL;

/* PWM Initialization */
void PWM_Init(PWM_HAL * const pwm);

/* Start PWM */
HAL_StatusTypeDef PWM_Start(PWM_HAL * const pwm);

/* Stop PWM */
HAL_StatusTypeDef PWM_Stop(PWM_HAL * const pwm);

/* Set PWM Duty Cycle */
void PWM_SetDutyCycle(PWM_HAL * const pwm, uint8_t duty);

/* Get PWM Duty Cycle */
uint8_t PWM_GetDutyCycle(const PWM_HAL * const pwm);
void PWM_ObjectConfig(PWM_HAL * const pwm,
                      TIM_HandleTypeDef *timer,
                      uint32_t channel,
                      uint32_t period);
```

- **PWM_Init()** – Initializes the PWM object.
- **PWM_ObjectConfig()** – Configures the timer, channel, and PWM period.
- **PWM_Start()** – Starts PWM output.
- **PWM_Stop()** – Stops PWM output.
- **PWM_SetDutyCycle()** – Sets the PWM duty cycle from 0% to 100%.
- **PWM_GetDutyCycle()** – Reads the current PWM duty cycle.

```c
/* Initialize PWM */
void PWM_Init(PWM_HAL * const pwm)
{
    if (pwm != NULL)
    {
        pwm->TimerHandle = NULL;
        pwm->Channel = 0U;
        pwm->Period = 0U;
    }
}
void PWM_ObjectConfig(PWM_HAL * const pwm,
                      TIM_HandleTypeDef *timer,
                      uint32_t channel,
                      uint32_t period)
{
    if ((pwm != NULL) && (timer != NULL))
    {
        pwm->TimerHandle = timer;
        pwm->Channel = channel;
        pwm->Period = period;
    }
}
```

M_Init() – Initializes the PWM object setting the TimerHandle, Channel, Period to default values.

M_ObjectConfig() – Configures the M object by assigning the timer dle, PWM channel, and timer period he object.

```c
HAL_StatusTypeDef PWM_Start(PWM_HAL * const pwm)
{
    HAL_StatusTypeDef status = HAL_ERROR;

    if ((pwm != NULL) && (pwm->TimerHandle != NULL))
    {
        status = HAL_TIM_PWM_Start(
                    pwm->TimerHandle,
                    pwm->Channel);
    }

    return status;
}

/* Stop PWM */
HAL_StatusTypeDef PWM_Stop(PWM_HAL * const pwm)
{
    HAL_StatusTypeDef status = HAL_ERROR;

    if ((pwm != NULL) && (pwm->TimerHandle != NULL))
    {
        status = HAL_TIM_PWM_Stop(
                    pwm->TimerHandle,
                    pwm->Channel);
    }

    return status;
}
```

- **PWM_Start()** – Starts PWM output on the configured timer channel using **HAL_TIM_PWM_Start().**
- **PWM_Stop()** – Stops PWM output on the configured timer channel using **HAL_TIM_PWM_Stop().**
- Both functions first check that the PWM object and timer handle are valid before performing the operation.

```c
/* Set PWM Duty Cycle */
void PWM_SetDutyCycle(PWM_HAL * const pwm, uint8_t duty)
{
    uint32_t compareValue;

    if ((pwm != NULL) && (pwm->TimerHandle != NULL))
    {
        if (duty > 100U)
        {
            duty = 100U;
        }

        compareValue =
            ((uint32_t)duty * (pwm->Period + 1U)) / 100U;

        __HAL_TIM_SET_COMPARE(
            pwm->TimerHandle,
            pwm->Channel,
            compareValue);
    }
}

/* Get PWM Duty Cycle */
uint8_t PWM_GetDutyCycle(const PWM_HAL * const pwm)
{
    uint32_t compareValue;
    uint8_t duty = 0U;

    if ((pwm != NULL) && (pwm->TimerHandle != NULL))
    {
        compareValue =
            __HAL_TIM_GET_COMPARE(
                pwm->TimerHandle,
                pwm->Channel);

        if (pwm->Period != 0U)
        {
            duty = (uint8_t)
                ((compareValue * 100U) /
                 (pwm->Period + 1U));
        }
    }
```

- **PWM_SetDutyCycle()** – Sets the PWM duty cycle between **0% and 100%** by calculating the required compare value and updating the timer's CCR register using __HAL_TIM_SET_COMPARE(). C
- **PWM_GetDutyCycle()** – Reads the current **CCR value** using __HAL_TIM_GET_COMPARE() and converts it into the corresponding PWM duty cycle percentage.
- Both functions check that the PWM object and timer handle are valid before performing the operation.

mian.cpp

```c
PWM_ObjectConfig(
    &PWM1,
    &htim3,
    TIM_CHANNEL_1,
    999U
);

PWM_ObjectConfig(
    &PWM2,
    &htim3,
    TIM_CHANNEL_2,
    999U
);

PWM_ObjectConfig(
    &PWM3,
    &htim3,
    TIM_CHANNEL_3,
    999U
);
```

These three function calls configure the three PWM channels of TIM3:

- PWM1 → Uses TIM3 Channel 1 (PA6) with a period of 999.
- PWM2 → Uses TIM3 Channel 2 (PA7) with a period of 999.
- PWM3 → Uses TIM3 Channel 3 (PB0) with a period of 999.

The same timer (htim3) is used for all three PWM outputs, allowing the LEDs to operate at the same PWM frequency while having independently controlled duty cycles.

```c
  /*-----------------------------------------------
   * Start PWM
   *-----------------------------------------------*/
  PWM_Start(&PWM1);
  PWM_Start(&PWM2);
  PWM_Start(&PWM3);

  /*-----------------------------------------------
   * Set fixed brightness for testing
   *-----------------------------------------------*/
  PWM_SetDutyCycle(&PWM1, 100U);
  PWM_SetDutyCycle(&PWM2, 100U);
  PWM_SetDutyCycle(&PWM3, 100U);

  /*-----------------------------------------------
   * Main Loop
   *-----------------------------------------------*/
  while (1)
  {
       uint16_t adcValue;
          uint8_t duty;

          adcValue = ADC_Read(&ADC);

          duty = (uint8_t)(
              ((uint32_t)adcValue * 100U) / 4095U
          );

          PWM_SetDutyCycle(&PWM1, duty);
          PWM_SetDutyCycle(&PWM2, duty);
          PWM_SetDutyCycle(&PWM3, duty);

          HAL_Delay(10U);
  }
```

PWM_Start(&PWM1), PWM_Start(&PWM2), and PWM_Start(&PWM3) start PWM output on TIM3 Channels 1, 2, and 3.

PWM_SetDutyCycle() initially sets all three LEDs to 100% brightness for testing.

ADC_Read(&ADC) reads the potentiometer value through the ADC, producing a value from 0 to 4095.

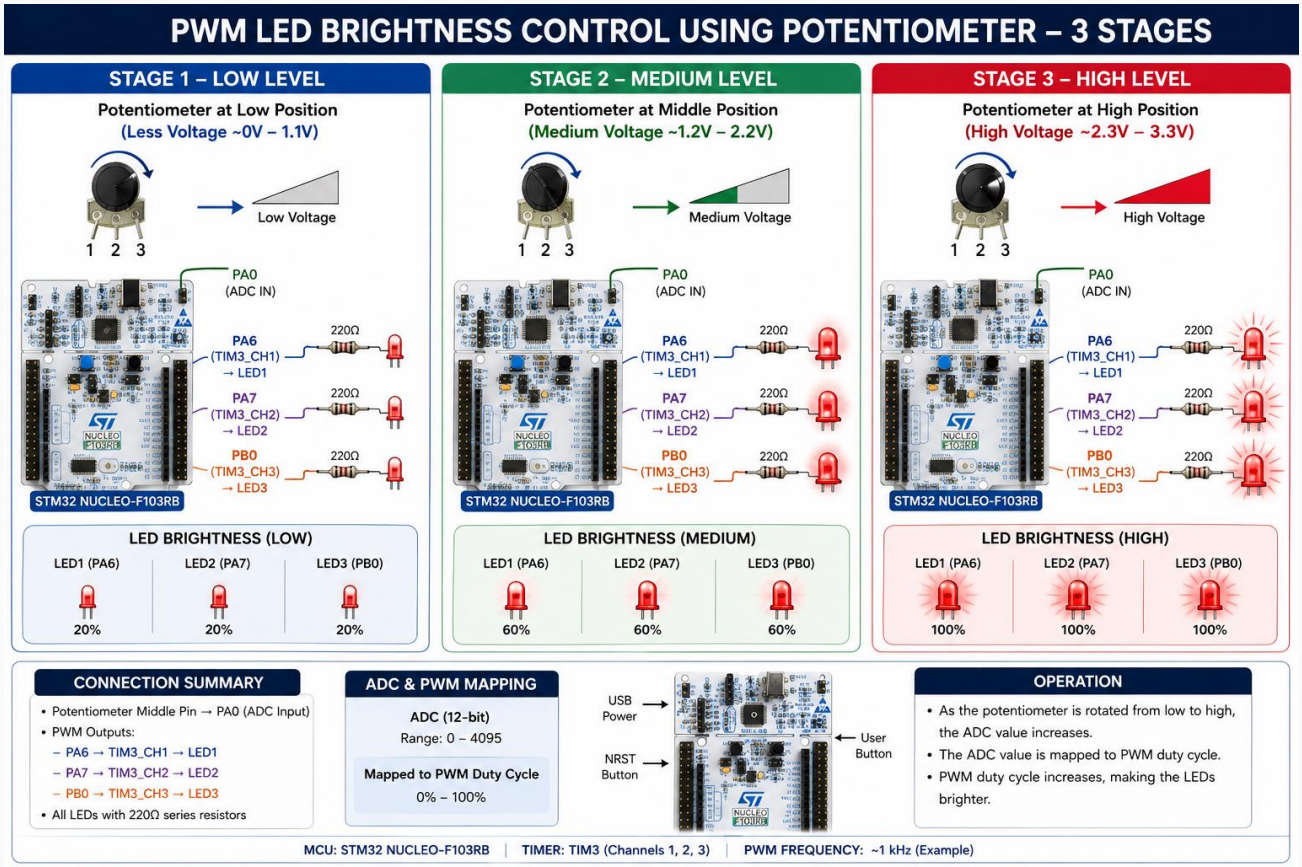
The ADC value is converted into a 0–100% duty cycle using:

$$Duty = \frac{ADC\ Value \times 100}{4095}$$

The calculated duty cycle is applied to all three PWM channels, so the three LEDs change brightness according to the potentiometer position.

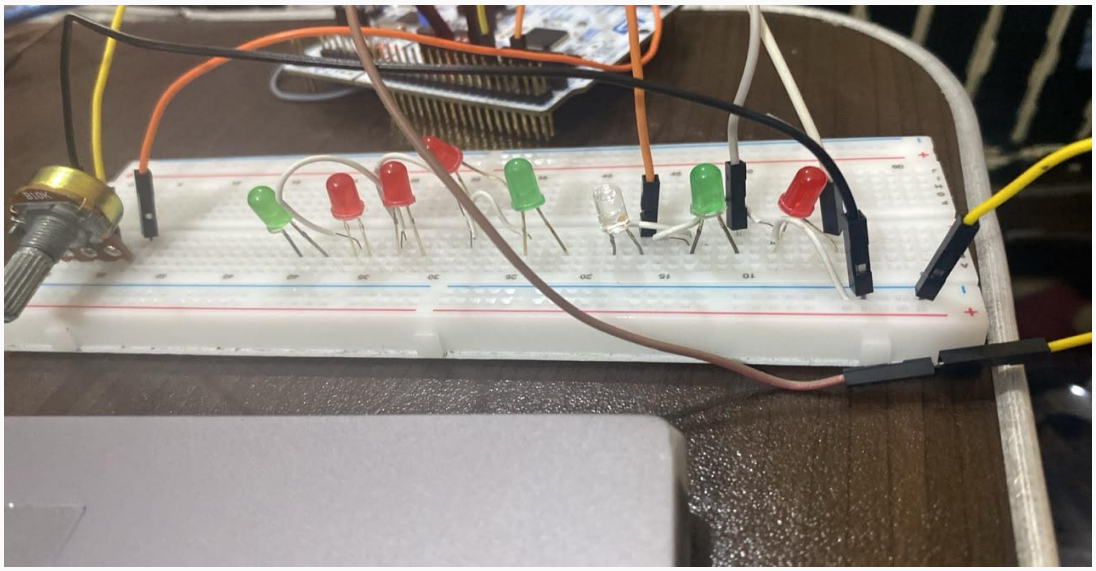
HAL_Delay(10U) adds a 10 ms delay before the next ADC reading and PWM update.

Program Output

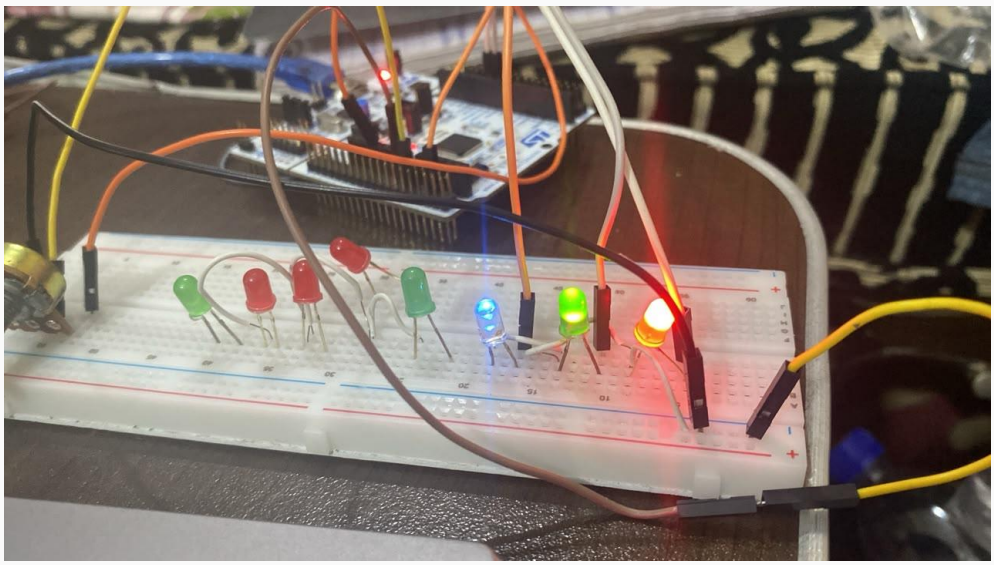


Stage 1

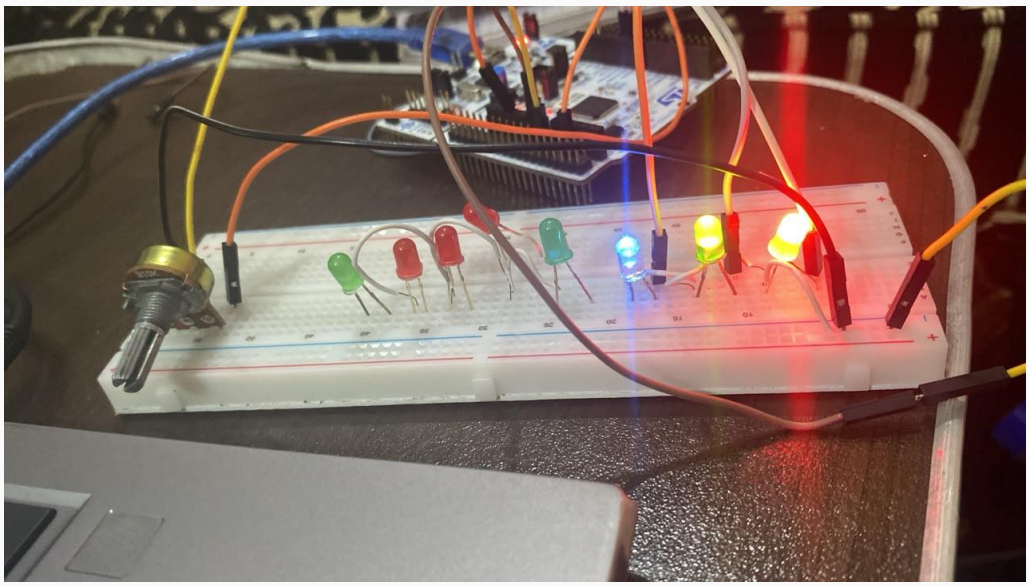
LED Brightness 0



Stage 2



Stage 3



Result

- The 10kΩ potentiometer successfully provided a variable analog voltage to PA0 (ADC1_IN0).
- The STM32F103RB ADC successfully converted the analog input into a 12-bit digital value from 0 to 4095.
- The ADC value was successfully mapped to a PWM duty cycle from 0% to 100%.
- TIM3 PWM channels successfully generated PWM signals through PA6, PA7, and PB0.
- The brightness of all three LEDs changed according to the potentiometer position: low, medium, and high brightness.
- The experiment successfully demonstrated ADC-based PWM control using the STM32 Nucleo-F103RB, providing practical understanding of ADC, timers, PWM, and LED brightness control.

Conclusion

The experiment successfully demonstrated ADC-based PWM control using the STM32 Nucleo-F103RB. The potentiometer provided a variable analog input, which was converted into a digital ADC value and mapped to a PWM duty cycle from 0% to 100%. TIM3 generated PWM signals to control the brightness of three LEDs. Overall, the experiment provided practical understanding of ADC, PWM, timers, duty-cycle control, and STM32 HAL programming.

Github Repositary:

https://github.com/Ahsan-web-hue/STM32F103RB-ADC-Based-PWM-LED-Brightness-Control.git